02 — Engineering & Operations Lessons (QCC)

Cập nhật 2026-10-31. Kinh nghiệm lập trình, môi trường, Kaggle, tổ chức thử nghiệm, lỗi đã gặp và cách sửa. Khoa học ở `01`, trạng thái/việc tiếp theo ở `03`.

1. Môi trường

Thành phần	Ghi chú
Python	`kickmix.so` của cuộc thi build cho **CPython 3.13** (container có 3.11/3.12/3.13; chỉ 3.13 import được). Kaggle image cũng là 3.13
kickmix	Có trên PyPI: `pip install kickmix==1.0.0` (wheel cp313 manylinux). Cho kết quả trùng tuyệt đối với bản trong dữ liệu cuộc thi (cost tới 10 chữ số)
Gói cần cài	`pip install kaggle scipy` (scipy cho LP/HiGHS). numpy 2.x có sẵn
Tài nguyên container	4 CPU, 15 GB RAM, ~30 GB đĩa trống
Thời gian chạy	verifier đầy đủ 200M shot: ~10 phút trên 4 core local, ~800 s trên Kaggle CPU. 1M shot: ~3 s
Dữ liệu cuộc thi	Không đưa lên git (luật cấm phát tán). Tải bằng `kaggle competitions download <comp> -f <path>`. Lệnh `files` phân trang, cần `--page-token` để xem trang 2 (verifiers)
Trang mô tả	`kaggle competitions pages` không trả nội dung. Dùng API: `KaggleApi().competition_list_pages(comp)` → `.content`

2. Kaggle

Tài khoản và quy tắc

- **Token:** lưu ở `~/.kaggle/tok_<username>` (chmod 600), không bao giờ commit. `nhatlong1103` là acc 1 (đã tham gia cuộc thi); `yuan1103` là acc 2 (chưa chấp nhận luật, nên không gắn được dữ liệu cuộc thi). `automation/run_kernel.py` tự chọn token theo tên acc trong `kernel-metadata.json`.
- **Chỉ nộp bài từ `nhatlong1103`:** luật "one account" cấm nộp từ nhiều acc.
- **Tính toán trên `yuan1103`:** phiên HMM chạy song song trên cùng repo đã đánh dấu `yuan1103` là dành cho người dùng (`automation/reserved_accounts.txt`), còn `nhatlong1103` thì phiên HMM vẫn dùng, nên slot CPU (5/acc) của acc đó có thể đã bị chiếm.
- **Nộp bài:** file **phải tên `submission.zip`**, nếu không API trả 400 `FAILED_PRECONDITION`; chi tiết lỗi chỉ thấy được khi gọi qua Python API. Chấm mất khoảng 45 phút; theo dõi bằng `qcc/wait_scores.sh` (chạy nền, poll 2 phút, thoát khi hết `PENDING`). Tối đa 5 lượt/ngày.
- **Zip nộp bài** gồm `invsqrt.kmb`, `dqi.kmb` (~540 MB, nén còn ~26 MB) và `dfthcsos/` (angle_k_* + submission.npz), tạo bằng `qcc/make_submission.py`.

Kernel verify (`qcc/kaggle_verify.py`)

- **Thiết kế:** script kernel tự chứa. Circuit (kmx text hoặc kmb base64) và verifier được nén lzma + base64, nhúng thẳng vào `main.py` (~64 KB/circuit). kickmix cài từ PyPI, nên bật `enable_internet`.
- Không dùng dataset, nhờ vậy không có đua version khi nhiều kernel cùng acc (bài học cũ của phiên HMM: hai fanout đè version dataset của nhau).
- Chạy được cả trên acc chưa tham gia cuộc thi.
- **Khởi chạy:** `--launch` gọi `automation/run_kernel.py` nền với stall = 300 s và 4 lần restart, đúng quy tắc người dùng đặt ("không có output ~5 phút thì huỷ và tạo phiên mới, tối đa 4 lần"). Dòng cuối log `STATUS=DONE:<state>`; kết quả nằm ở `runs/<slug>/out/results.json`.
- **Biến môi trường:** `QCC_RUNS` (thư mục runs), `QCC_VERIFIER` (đường dẫn verifier local).
- **Dọn dẹp:** output kernel có cả thư mục `c/` (bản sao circuit); xoá sau khi kéo về để đỡ chỗ.

3. Mã và thiết kế thử nghiệm

Công cụ	Ý tưởng	Lưu ý hiệu năng
`ksim.py`	Mỗi shot là một làn bit. Một qubit / bit cổ điển là `uint64[W]` (W = [shots/64]), một cổng là một phép numpy. Pha tách thành parity nửa vòng (bitplane), bộ đếm 1/8 vòng (int64/shot), và các vector bit của từng rotation	Toàn miền 3.1M x trong ~15 s. Hook `probe(idx, op, q, mask)` trước mỗi Toffoli; `snap_at` chụp (qubit, half, eighth, #rot); `track_life` cho vòng đời qubit
`refit_angles.py`	LP: max t với |e0 + B·d| ≤ tol − t. Cutting plane trên các x tệ nhất	**Đơn vị 2^−40 vòng**: sai số ~1e−11 nhỏ hơn dung sai khả thi tuyệt đối của HiGHS (1e−7), dùng đơn vị vòng thì LP "thành công" nhưng sai. Target/tol được cache (isqrt 3.1M số mất vài giây)
`peephole.py`, `redundancy.py`	Giải GF(2) trên mẫu shot, xác nhận trên toàn miền	Khử Gauss numpy ổn với ~60 cột. Với 1 900 cột (bậc 2, 61 dây) mỗi lần giải tốn hàng chục giây, nên 134 CCZ không xong trong 50 phút. Thay bằng lọc băm (so cặp AND trên 96 word mẫu)
`probes/`	`block.py` (khối có phải oracle pha sạch không), `ccz_fast.py`, `xbits_run.py`	Tham số là thư mục chứa `best_pub.kmx` (bản kmx của circuit tốt nhất)

Nguyên tắc kiểm chứng.

- Mọi "luôn bằng 0 / luôn bằng hàm f" phải kiểm trên **nhiều seed HMR**: dây có thể phụ thuộc đồng xu qua CX/CZ có điều kiện. Ví dụ peephole: 51 ứng viên ở seed 0 chỉ còn 42 ổn định sau 3 seed.
- Kết quả cuối chỉ tin khi verifier gốc PASS đủ 200M shot.

4. Lỗi đã gặp và cách sửa

Lỗi	Nguyên nhân	Sửa
LP báo margin âm / `math domain error`	HiGHS dùng dung sai tuyệt đối 1e−7 ≫ thang 1e−11	Đổi đơn vị sang 2^−40 vòng; `rotation_cost` trả inf khi eps ≤ 0
OOM (exit 137) khi chụp trạng thái	`snap_at` chép cả mảng bit cổ điển: 30 001 bit × W word ≈ 3.9 GB	Chỉ chụp qubit + pha
`ValueError: last axis must be contiguous`	`.view(uint8)` trên slice không liên tục	`np.ascontiguousarray` trước `.view`
Mất output của job dài	`... | tail -20` giữ hết output tới khi job kết thúc; job bị `timeout` giết thì không còn gì	Ghi thẳng ra file log (`> x.log 2>&1`), `print(flush=True)`
Peephole bỏ sót CCZ	Chỉ xét a·b (2 điều khiển đầu)	Xét a·b·c cho CCZ (`probes/ccz_fast.py`)
Push bị từ chối / "no merge base"	Phiên HMM cùng push lên `hmm-navigation-2`; clone nông	`git checkout -B hmm-navigation-2 origin/hmm-navigation-2` + cherry-pick. Về sau luôn `git fetch && git rebase origin/hmm-navigation-2` trước khi push
Nộp bài lỗi 400	Tên file khác `submission.zip`	Đặt đúng tên
Tác vụ chờ nền bị giết	Trần thời gian chạy nền của Claude; các waiter `until grep ...` chờ một job đã chết	Dừng waiter khi job gốc chết; dùng timeout tối đa và một waiter cho mỗi sự kiện

5. Bài học

- **Đo trước khi xây.** Giải phẫu ngân sách bằng simulator (mất 1 giờ) cho thấy lineage đã sát tối ưu, tránh được một đêm dựng generator từ đầu chắc chắn thua.
- **Dữ liệu cộng đồng là nguồn lớn nhất.** Manifest `manwithacat/vqcc-apache-public-circuits` có mọi circuit public kèm cost local. README `mrmeows/qcc-1375-open-circuits` ghi lại những gì đã thử/thất bại.
- Dataset Kaggle tự giải nén zip khi upload, nên file `sources/*.zip` không tải được; phải tải từng file con. Liệt kê đầy đủ bằng `KaggleApi().dataset_list_files(..., page_token=...)` có phân trang.
- **Ưu tiên kiểm tra rẻ.** Thứ tự nên đi: affine → bậc 2 → hỗ trợ nhỏ. Lọc trên mẫu, xác nhận trên toàn miền.
- **Đừng tin một seed.** Xem nguyên tắc kiểm chứng ở mục 3.